

Лабораторная №13. Итоговое проектное задание по вариантам

Цель работы

Самостоятельно спроектировать и реализовать распределённую систему, объединяющую одноплатный компьютер Lichee RV Dock, микроконтроллер Arduino Uno и графическое приложение на ПК. Транспорт между компонентами — протокол MQTT (брокер Mosquitto на Lichee).

Оборудование и исходные данные

- оборудование, указанное в полученном варианте;
- Lichee RV Dock и Arduino Uno для вариантов с аппаратной связкой;
- компьютер с необходимыми средствами разработки;
- изолированная учебная сеть с MQTT-брокером Mosquitto на Lichee;
- результаты и примеры лабораторных №7–12.

Порядок выполнения

1. Каждый студент получает **один из десяти вариантов** задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для C-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений

Внимание

Полученный вариант выполняется полностью: обязательны все перечисленные в нём компоненты, функции и ключевые проверки. Пункты, явно помеченные как дополнительные или опциональные, не заменяют основную часть варианта.

Опасность повреждения

Во всех вариантах с физическим соединением Arduino Uno и Lichee RV Dock линии SPI должны проходить через преобразователь логических уровней 5↔3,3 В: сторона HV питается от Arduino 5 В, сторона LV — от Lichee 3,3 В. Arduino, Lichee, преобразователь и периферия должны иметь общий GND; прямое попадание сигнала 5 В на GPIO Lichee недопустимо.

Охват лабораторных: №7 (SPI), №8 (I2C OLED), №10 (Arduino), №11 (Arduino + BME280 + SPI), №12 (MQTT).

Вариант 1. Метеостанция — сбор и визуализация данных с датчика BME280

Описание

Собрать распределённую систему, в которой данные с датчика BME280 (температура, влажность, давление) передаются от Arduino к одноплатнику Lichee, далее через MQTT на ПК и отображаются в реальном времени в графическом интерфейсе.

Цепочка

BME280 —I2C→ Arduino Uno —SPI(slave)→ Lichee RV Dock —MQTT→ ПК (GUI)

Требования

1. **Arduino + датчик (лаб. №11).** Прошивка Arduino читает BME280 по I2C (библиотека GyverBME280) и передаёт три значения float (температура, °C; влажность, %; давление, гПа) по SPI в режиме ведомого устройства на Lichee. Протокол — серия однобайтовых транзакций.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Python-скрипт на Lichee принимает данные от Arduino через /dev/spidev1.0, распаковывает `struct.unpack("<fff", ...)` и публикует их в MQTT-топик `sensors/bme280` в формате JSON:

```
{"temperature": 25.1, "humidity": 48.3, "pressure": 1004.5, "timestamp":  
  ↪ "2026-05-04T12:34:56"}
```
3. **I2C OLED (лаб. №8).** Дополнительно — дублировать данные на OLED-экран SSD-1306 через I2C на Lichee (три строки: температура, влажность, давление).

Проверка

Дополнительная демонстрация. OLED в варианте 1 служит дополнительным каналом локальной индикации. Подключайте его к I2C Lichee с питанием 3,3 В; отсутствие OLED не должно нарушать SPI-приём и MQTT-публикацию.

4. **ПК — GUI-приложение.** Python-приложение, подключающееся к MQTT-брокеру на Lichee и отображающее:
 - Текущие значения трёх параметров — крупным шрифтом в отдельных блоках

- Три графика (matplotlib, встроенный в GUI) с историей за последние 60 секунд (ось X — время, ось Y — значение)
- Статус подключения: индикатор (зелёный/красный кружок) — получает ли система данные. Статус менять по тайм-ауту: если данные не поступали более 5 секунд — «нет данных»

Ключевые точки проверки

- Физически подключённый BME280 выдаёт реалистичные значения
- При остановке MQTT-публикации на Lichee GUI не позднее чем через 5 секунд показывает «нет данных» (красный индикатор)
- При отключении Arduino от Lichee — аналогично
- Графики обновляются плавно, история не теряется при свёртывании окна

Вариант 2. Пульт управления светодиодами через MQTT и SPI

Описание

Через графический интерфейс на ПК управлять светодиодами на мезонинной плате Arduino, передавая команды по цепочке MQTT → Lichee → SPI → Arduino.

Цепочка

ПК (GUI) —MQTT→ Lichee RV Dock —SPI(master)→ Arduino Uno —GPIO→
↔ LED-мезонин

Требования

1. **Arduino — приём команд по SPI (лаб. №10, №11).** Прошивка работает в режиме SPI-slave и управляет четырьмя светодиодами на пинах 3, 5, 6 и 9 мезонинной платы. Пин 10 зарезервирован аппаратным SPI как вход SS и для управления светодиодом не используется. Команды приходят от Lichee-мастера. Протокол команд студент проектирует самостоятельно, например байт команды и байт данных или четыре бита для четырёх светодиодов.

Поддерживаемые операции для каждого светодиода:

- включить / выключить (индивидуально)
- «бегущий огонёк» вперёд (пины 3→5→6→9→3)
- «бегущий огонёк» назад (пины 9→6→5→3→9)
- остановить анимацию

Скорость бегущего огонька задаётся параметром (задержка в миллисекундах).

2. **Lichee — MQTT-подписчик + SPI-мастер (лаб. №12).** Python-скрипт подписывается на MQTT-топик `led/command`, получает JSON-команды и транслирует их в SPI-транзакции к Arduino.

Формат JSON-команды (примеры):

```
{"pin": 3, "action": "on"}
{"pin": 5, "action": "off"}
{"action": "animation", "direction": "forward", "delay_ms": 150}
{"action": "stop"}
```

3. **ПК — GUI-приложение.** Окно с элементами управления:

- четыре тумблера для индивидуальных светодиодов с подписями «LED 3», «LED 5», «LED 6», «LED 9»;
- Кнопка «Бегущий вперёд» и кнопка «Бегущий назад»
- Слайдер задержки анимации (50–500 мс)
- Кнопка «Стоп»
- При изменении любого элемента GUI публикуется соответствующая MQTT-команда

Ключевые точки проверки

- Каждый тумблер индивидуально включает/выключает свой светодиод
- Бегущий огонёк работает в обоих направлениях
- Смена задержки на слайдере меняет скорость анимации
- «Стоп» останавливает анимацию и оставляет текущее состояние светодиодов

Вариант 3. Панель системного мониторинга одноплатника

Описание

Создать дашборд (панель мониторинга), отображающий в реальном времени системные метрики одноплатника Lichee: загрузку CPU, использование RAM, занятость корневого раздела, сетевой трафик. Метрики собираются на Lichee и публикуются по MQTT. GUI на ПК отображает их в виде приборных панелей и графиков.

Цепочка

Lichee RV Dock (/proc, /sys) —MQTT—> ПК (GUI)

Требования

1. **Lichee — сборщик метрик (лаб. №12).** Python-скрипт-издатель, публикующий в MQTT-топик `lichee/stats` JSON-пакет раз в 2 секунды:

```
{
  "cpu_percent": 12.3,
  "ram_percent": 45.7,
  "disk_percent": 32.1,
  "rx_bytes_per_sec": 1024,
  "tx_bytes_per_sec": 512,
  "uptime_seconds": 12345,
  "hostname": "lichee-rv",
  "kernel": "6.16.0"
}
```

Источники данных:

- CPU: /proc/stat
- RAM: /proc/meminfo
- Диск: os.statvfs('/')
- Сеть: /sys/class/net/<интерфейс>/statistics/rx_bytes, tx_bytes (дельта между опросами)
- Uptime: /proc/uptime
- Hostname: socket.gethostname()
- Версия ядра: os.uname().release

2. **Дополнительно.** Написать сборщик метрик на языке C с использованием только системных вызовов и файлов /proc, /sys. Скомпилировать под RISC-V (кросс-компиляция с x86_64). По быстродействию сравнить с Python-версией.

Проверка

Дополнительное сравнение. C-сборщик в варианте 3 выполняется после готовой Python-версии. Используйте одинаковый набор метрик и интервал публикации, отдельно зафиксируйте размер процесса, загрузку CPU и размер исполняемого файла.

3. **ПК — GUI-приложение.** Интерфейс дашборда содержит:

- 4 «приборных панели» (gauge / стрелочный индикатор) для CPU, RAM, диска, сети. Каждая панель показывает текущее значение в процентах и имеет цветовое кодирование (зелёный < 50%, жёлтый 50–80%, красный > 80%)
- Строка статуса в нижней части окна: uptime, hostname, версия ядра
- График истории (опционально): загрузка CPU и RAM за последние 60 секунд
- Кнопка «Обновить сейчас» — публикует команду в lichee/cmd ({"action": "refresh"})

Ключевые точки проверки

- Все четыре панели обновляются и показывают адекватные значения
- При нагрузке на Lichee (запуск stress или dd) значения меняются
- Кнопка «Обновить сейчас» вызывает немедленную публикацию
- Строка статуса показывает реальные uptime и hostname

Вариант 4. Генератор сигналов с управлением по MQTT — практическая верификация анализатором

Описание

С графического интерфейса на ПК задать параметры цифрового сигнала (меандра). Lichee генерирует сигнал на GPIO-пине. Студент подключает логический анализатор к этому пину, захватывает сигнал в PulseView и подтверждает соответствие заданным параметрам.

Цепочка

ПК (GUI) —MQTT→ Lichee (libgpiod2) —GPIO→ Логический анализатор (fx2lafw)
↔ —USB→ ПК (PulseView)

Требования

1. **Lichee — генератор сигнала (лаб. №9).** Python-скрипт подписывается на MQTT-топик `gpio/control`, принимает JSON-команды и управляет GPIO-пином (PE16, линия 144) через `libgpiod2`.

Формат JSON-команды:

```
{"action": "start", "gpio_line": 144, "freq_hz": 1000, "duty_percent": 50}  
{"action": "stop"}
```

После получения "start" скрипт в отдельном потоке генерирует меандр. При "stop" поток останавливается, пин переводится в низкий уровень. При повторном "start" без "stop" параметры обновляются без разрыва сигнала.

2. **ПК — GUI-приложение.** Окно с элементами управления:

- Выпадающий список с доступными GPIO-линиями Lichee (линии 144, 145, 146 — PE16, PE17, PE18)
- Поле ввода частоты (Гц) — от 1 до 1000 Гц, с проверкой корректности ввода. Значение по умолчанию: 100 Гц
- Слайдер или поле ввода скважности (%) — от 1 до 99%. По умолчанию: 50%
- Кнопка «Старт» и кнопка «Стоп»

- Предпросмотр сигнала — на Canvas отрисовывается визуализация меандра (два периода) с подписями периода и длительности уровней

3. **Верификация (лаб. №9).** Подключить логический анализатор (fx2lafw) к заданному GPIO-пину Lichee. Запустить захват в PulseView (частота дискретизации $\geq 4\times$ от частоты сигнала, но не менее 12 МГц). Измерить период и скважность сигнала и сравнить с заданными в GUI значениями.

Ключевые точки проверки

- Сигнал на GPIO появляется при нажатии «Старт» и пропадает при «Стоп»
- Частота и скважность, измеренные в PulseView, соответствуют заданным (погрешность в пределах 5% во всём заданном диапазоне)
- Изменение параметров через GUI во время работы генератора обновляет сигнал без остановки
- Предпросмотр на Canvas корректно отображает два периода меандра с верными метками времени

Вариант 5. Светодиодный проигрыватель мелодий

Описание

Реализовать визуальный проигрыватель мелодий, в котором на ПК в графическом редакторе задаётся «партитура» — последовательность включений светодиодов во времени. Мелодия передаётся по цепочке MQTT → Lichee → SPI → Arduino и проигрывается на четырёх светодиодах мезонинной платы, подключённых к пинам 3, 5, 6 и 9. Пин 10 зарезервирован как вход SS аппаратного SPI.

Цепочка

ПК (GUI-редактор) —MQTT→ Lichee RV Dock —SPI(master)→ Arduino Uno
↔ —GPIO→ 4 LED

Ключевое проектное решение

Arduino получает только один байт — битовую маску четырёх светодиодов (биты 0–3). Вся логика темпа (BPM) и таймингов размещается на Lichee, что минимизирует Arduino-код и упрощает SPI-протокол до однобайтовых транзакций.

Требования

1. **Arduino — прошивка SPI-slave (лаб. №10, №11).** Минимальный скетч в режиме SPI-slave. Принимает один байт от Lichee-мастера и устанавливает пины в соответствии с битами принятого байта. SPI: режим slave, пины MISO(12)/MOSI(11)/SCK(13)/SS(10). При получении 0x00 — гасит всё.
2. **Lichee — MQTT-подписчик + SPI-транслятор (лаб. №12).** Python-скрипт, подписывающийся на MQTT-топик melody/command. При "play" — сохраняет массив шагов и BPM, вычисляет длительность одного шага и в отдельном потоке отправляет битовые маски по SPI с нужной задержкой. После последнего шага отправляет 0x00 и публикует статус "stopped". SPI: /dev/spidev1.0, mode 0, скорость 100 кГц.
3. **ПК — GUI-редактор мелодии.** Окно приложения содержит:
 - редактор на Canvas с сеткой: четыре строки для LED 3, 5, 6 и 9, столбцы — шаги, не более 64; щелчок по ячейке переключает состояние;
 - Слайдер темпа (BPM): 60–240, по умолчанию 120
 - Кнопки Play, Stop
 - Выпадающий список «Загрузить пример»: 3 встроенных мелодии
 - Кнопка «Сохранить» / «Загрузить» — JSON-файл
 - Индикация проигрывания: вертикальная красная линия движется по текущему шагу

MQTT-команды в топик melody/command:

```
{"action": "play", "bpm": 120, "melody": [[1,0,0,1], [0,0,1,0], ...]}  
{"action": "stop"}
```

Ключевые точки проверки

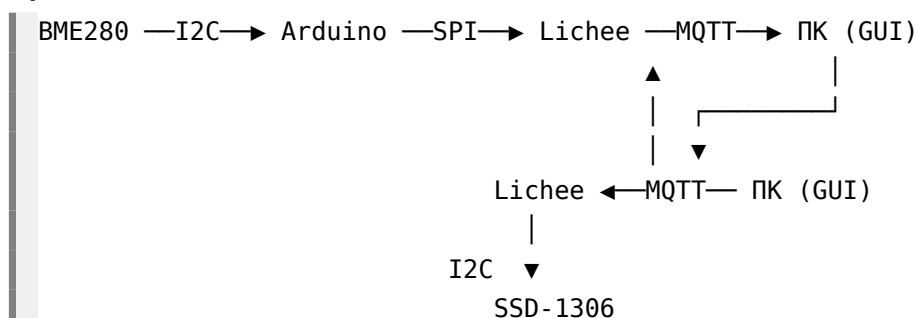
- При Play светодиоды загораются в точном соответствии с рисунком в редакторе
- Красная линия в GUI движется синхронно с физическими светодиодами
- При Stop всё гаснет
- Изменение BPM на слайдере ускоряет/замедляет проигрывание
- Сохранение и загрузка JSON-файла работают корректно

Вариант 6. Система мониторинга с оповещением на I2C OLED

Описание

Расширение варианта №1 (Метеостанция). Данные с датчика BME280 идут по цепочке Arduino → SPI → Lichee → MQTT → GUI на ПК. Дополнительно оператор задаёт в GUI пороговые значения для каждого параметра. При превышении порога GUI сигнализирует тревогу. Оператор жмёт кнопку «Оповестить на OLED» — по обратному каналу MQTT → Lichee на I2C-экран SSD-1306 выводится тревожное сообщение с текущим значением.

Цепочка



Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C (GyverBME280), передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Принимает данные по SPI, публикует JSON в топик sensors/bme280.
3. **Lichee — подписчик на тревоги (лаб. №8, №12).** Отдельный Python-скрипт, подписывающийся на топик alert/oled. При получении сообщения выводит текст на SSD-1306 через I2C (luma-oled). Через 5 секунд автоматически очищает экран.
4. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов: T_max, H_max, P_min, P_max со значениями по умолчанию
 - Индикация тревоги: при превышении порога строка параметра подсвечивается красным и мигает
 - Кнопка «Оповестить на OLED»: активна только когда есть активная тревога
 - Сброс тревоги: когда значение возвращается в пределы порога + гистерезис

Ключевые точки проверки

- При превышении порога GUI подсвечивает соответствующую строку красным
- Кнопка «Оповестить на OLED» активна только при тревоге; при нажатии сообщение появляется на физическом экране SSD-1306
- Через 5 секунд тревожное сообщение удаляется и экран очищается
- При возврате значения в норму тревога сбрасывается с гистерезисом

Вариант 7. Генератор загрузочных образов — консольный сборщик SD-карты

Описание

Создать консольное приложение на Python, которое из готовых компонентов автоматически собирает загрузочную SD-карту для Lichee RV Dock с заданной конфигурацией. Готовые ядра, Device Tree Blob, rootfs и загрузчик предоставляются преподавателем.

Цепочка

ПК (Python-скрипт) — fdisk/dd/mkfs/cp → SD-карта

Требования

Скрипт `build_image.py` с интерфейсом командной строки:

Опасность повреждения

Команды `fdisk`, `dd` и `mkfs` необратимо уничтожают данные на выбранном устройстве. Сначала выполните `--dry-run`, который обязан только вывести план без запуска команд записи; затем повторно проверьте имя блочного устройства, отсутствие смонтированных разделов и запросите явное подтверждение перед реальной записью.

```
python3 build_image.py --device /dev/sdX --config spi [--dry-run]
```

Конфигурация	Ядро	Device Tree	Поддерживаемые интерфейсы
basic	Image	sun20i-d1-lichee-rv-dock.dtb	WiFi
spi	Image-spi	sun20i-d1-lichee-rv-dock-spi.dtb	WiFi + SPI

Конфигурация	Ядро	Device Tree	Поддерживаемые интерфейсы
i2c	Image-i2c	sun20i-d1-lichee-rv-dock-i2c.dtb	WiFi + I2C
full	Image-spi	.dtb с SPI + I2C	WiFi + SPI + I2C

Действия программы (по порядку):

1. Проверка: `--device` существует, является блочным устройством и не примонтирован
2. Разбивка: `fdisk` создаёт таблицу разделов DOS (первый раздел 100 МБ FAT32, второй — остаток ext4)
3. Запись загрузчика: `dd if=u-boot-sunxi-with-spl.bin of=<device> bs=1k seek=8`
4. Форматирование: `mkfs.vfat -F 32 <device>1, mkfs.ext4 <device>2`
5. Копирование файлов на BOOT и распаковка rootfs
6. `sync` и уведомление о завершении

Вывод программы — человекочитаемый лог каждого шага:

```
[INFO] Checking device /dev/sdb ... OK (not mounted)
[INFO] Selected config: spi
[INFO] Step 1/6: Partitioning ...
[INFO] Step 2/6: Writing U-Boot (seek=8) ...
...
[INFO] Done. SD card is ready at /dev/sdb
```

Ключевые точки проверки

- Карта, собранная с конфигурацией `basic`, загружается, работает WiFi
- С конфигурацией `spi` — загружается, WiFi и SPI работают (проверить `ls /dev/spidev*`)
- С конфигурацией `i2c` — загружается, WiFi и I2C работают (проверить `ls /dev/i2c-*`)
- `--dry-run` выводит план, но не трогает SD-карту
- При попытке записать на примонтированное устройство — ошибка

Вариант 8. Система мониторинга со звуковой сигнализацией

Описание

Расширение варианта №1 (Метеостанция). При превышении порогового значения GUI автоматически воспроизводит звуковую сирену. Характер

звука настраивается для каждого измеряемого параметра индивидуально. Оператор может квитировать тревогу кнопкой.

Цепочка

BME280 —I2C→ Arduino —SPI→ Lichee —MQTT→ ПК (GUI + динамик)

Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Идентично варианту №1.
3. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов (T_max, H_max, P_min, P_max)
 - Настройки сирены для каждого параметра: тип звука (прерывистый быстрый/медленный/непрерывный/WAV-файл), частота тона (Гц)
 - Автоматическое воспроизведение сирены при превышении порога
 - Кнопка «Отключить сирену» (квитирование)
 - Лог тревог (таблица: Время, Параметр, Значение, Порог) + экспорт в CSV

Генерация звука — через стандартную библиотеку Python wave (без дополнительных зависимостей), воспроизведение через aplay.

Ключевые точки проверки

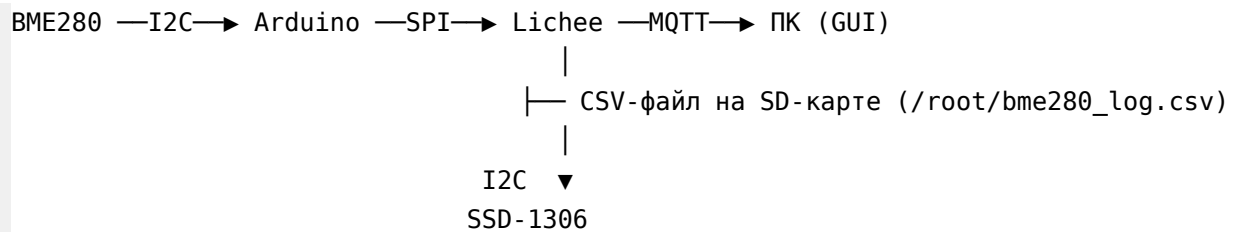
- При превышении порога GUI подсвечивает параметр красным и издаёт звуковую сирену
- Характер звука разный для разных параметров
- Кнопка «Отключить сирену» прекращает звук
- Лог тревог пополняется при каждом превышении; экспорт в CSV — корректный

Вариант 9. Самописец данных с хранением на SD-карте

Описание

Собрать систему, в которой данные с датчика BME280 накапливаются на SD-карте одноплатника и одновременно доступны для live-мониторинга через MQTT. GUI на ПК отображает текущие значения и позволяет загрузить историю измерений.

Цепочка



Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C, передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + логгер + MQTT (лаб. №7, №8, №12).** Python-скрипт:
 - Принимает данные от Arduino через `/dev/spidev1.0`
 - Дописывает строку в CSV-файл `/root/bme280_log.csv`:

```
2026-06-24T12:34:56,25.1,48.3,1004.5
```
 - Параллельно публикует текущие данные в MQTT-топик `sensors/bme280` (JSON)
 - Дублирует текущие значения на OLED SSD-1306
3. **ПК — GUI-приложение.** Окно с элементами:
 - Текущие значения (температура, влажность, давление)
 - График реального времени (данные из MQTT)
 - Кнопка «Загрузить историю» — по SCP или через HTTP качает CSV-файл с Lichee и отображает историю на отдельном графике
 - Строка статуса: общее количество записей в логе, дата первой и последней записи

Внимание

Для загрузки истории выберите один способ и опишите его в отчёте: SCP с аутентификацией или HTTP с доступом только в учебной сети. Не встраивайте пароль в исходный код и не публикуйте каталог `/root` целиком через HTTP.

Ключевые точки проверки

- Данные пишутся в CSV непрерывно, даже если GUI не запущен (проверить: запустить логгер, отключить ПК на минуту, снова подключиться — история сохранена)
- При перезагрузке Lichee файл лога сохраняется (SD-карта)
- График истории загружается корректно, временные метки соответствуют реальным

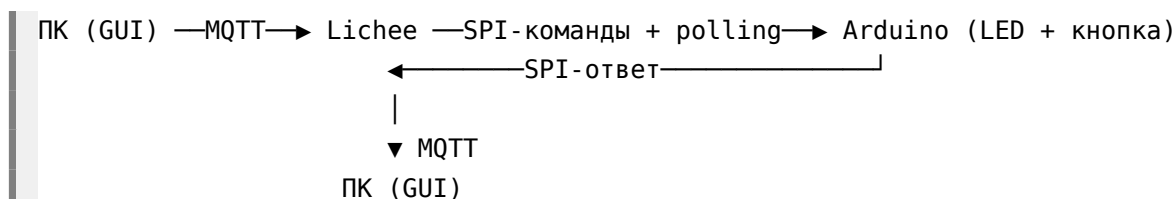
- Текущие значения на OLED обновляются синхронно с MQTT-поток

Вариант 10. Игра на реакцию

Описание

Реализовать игру на измерение времени реакции человека. ПК отправляет команду начала раунда. Lichee ждёт случайную задержку, затем зажигает светодиод на Arduino и одновременно публикует метку времени в MQTT. Игрок должен как можно быстрее нажать кнопку на Arduino, после чего вычисляется время реакции и отображается на GUI ПК.

Цепочка



Требования

Опасность повреждения

Нажатие кнопки обнаруживается локальным прерыванием Arduino, но передаётся на Lichee только как байт ответа при периодическом опросе SPI мастером. Не соединяйте кнопку или выход Arduino отдельной линией с GPIO-прерывания Lichee: прямая линия с уровнем 5 В опасна для Lichee.

1. Arduino — SPI-slave + локальное прерывание (лаб. №10, №11).

Прошивка:

- Режим SPI-slave: принимает однобайтовую команду от Lichee
- Команда 0x01 — зажечь LED на пине 5
- Команда 0x00 — погасить LED
- Команда 0x02 — опросить состояние кнопки без изменения LED
- Кнопка подключена к пину 2 (прерывание FALLING): при нажатии отправляет байт 0xFF в SPI-буфер (мастер при следующей транзакции считает его)

2. Lichee — ведущий игры (лаб. №12). Python-скрипт, подписывающийся на MQTT-топик game/start:

- При получении {"action": "round"}:
 - Начинает опрашивать Arduino командой 0x02 и ждёт случайную задержку 1-5 секунд; нажатие кнопки в этот интервал фиксирует как фальстарт

- Отправляет 0x01 по SPI (зажечь LED на Arduino)
- Фиксирует время `t0 = time.time()`
- Публикует `{"event": "go", "timestamp": t0}` в `game/events`
- Продолжает циклически отправлять команду 0x02 короткими SPI-транзакциями с частотой около 100 Гц и проверять байт ответа Arduino
- При получении 0xFF от Arduino фиксирует время реакции `dt = time.time() - t0` и публикует в `game/result`
- Отправляет 0x00 (погасить LED)

3. ПК — GUI-приложение. Окно с элементами:

- Кнопка «Новый раунд» — отправляет `{"action": "round"}` в `game/start`
- Крупное отображение времени реакции (мс) после завершения раунда
- Таблица последних 10 попыток (№, время реакции, результат)
- Лучший результат (минимальное время реакции за сессию)
- Индикатор состояния: «Ждите...», «ЖМИ!», «Результат»

Ключевые точки проверки

- Задержка перед зажиганием LED случайна и разная в каждом раунде
- Время реакции измеряется в миллисекундах (типичные значения 150–400 мс)
- Таблица результатов обновляется после каждого раунда
- При нажатии кнопки до зажигания LED (фальстарт) фиксируется как ошибка и не засчитывается
- GUI корректно отображает все состояния игры

Контрольные вопросы

1. Какую роль в итоговом проекте играет MQTT-брокер Mosquitto и почему он выбран в качестве транспортного протокола?
2. Почему в заданиях на связку Arduino→Lichee используется SPI, а не UART или I2C? Какие преимущества это даёт?
3. Каковы основные архитектурные паттерны, повторяющиеся во всех вариантах итогового задания?
4. Какие меры следует предусмотреть для обеспечения надёжности распределённой системы при возможных отказах компонентов?
5. Как организовать тестирование и отладку распределённой системы, когда все компоненты работают на разном оборудовании?

Порядок сдачи

1. Студент получает один из десяти вариантов задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для C-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений
6. Отправляет отчёт в согласованном формате до установленного срока.

Требования к отчёту

Отчёт должен содержать:

- цель и формулировку полученного варианта;
- архитектуру системы и схему аппаратных соединений;
- описание протоколов и форматов сообщений;
- исходные тексты компонентов и инструкции по сборке и запуску;
- результаты ключевых проверок варианта;
- ответы на контрольные вопросы;
- краткий вывод.